OPENLEARN V2 · 第三方插件开发

用插件开发 Skill
写一个真正能上课的第三方插件

本手册把 `openlearn-next-plugin-dev` 这个 AI 插件开发技能，拆成一条可复现的工程流水线：探活 → 判通道 → 生成骨架 → 写代码 → 测护栏 → 打包 → 安装 → 课堂验收。全文所有截图与终端输出，都是在真实平台（openlearn-next v0.7.10）上跑出来的一个第三方插件「课堂便签墙」的完整过程。

实测环境：openlearn-next v0.7.10 · @openlearn/plugin-sdk 3.8.0 · Node 22

完整周期：约 45 分钟（含读文档）；首个可用插件约 20 分钟

目录 CONTENTS

1. Skill 是什么、能替你做什么
2. 开工前的三件事（探活 / 基线 / 需求）
3. 第 1 步：一键生成项目骨架
4. 第 2 步：P0 门禁，判 UI 通道
5. 第 3 步：写后端（manifest 与三件套）
6. 第 4 步：写可见界面（沙箱 view）
7. 第 5 步：铺测试护栏（L0–L3）
8. 第 6 步：打包成可安装的 ZIP
9. 第 7 步：后台上传与授权
10. 第 8 步：课堂验收与交付清单
11. 踩坑清单（4 个真实翻车点）
12. 报错速查表与附录

SKILL

SECTION 01

一、这个 Skill 是什么，能替你做什么

`openlearn-next-plugin-dev` 是一个**装在 AI 编程助手里的插件开发技能**。 它不是 SDK，也不是模板集合，而是「开发 OpenLearn 第三方插件时该知道的一切」的知识包： 模板代码、探活脚本、契约静态检查器、故障归因速查表、以及一套踩坑清单。 典型用法是让 AI 助手加载这个技能后，按它的流程帮你把一个想法变成可安装的 ZIP。

5	4	9	20+
套可生成的项目模板 server-only / full-stack / frontend-only / sandbox-view / inline-react	层测试护栏 L0 静态契约 → L1 契约 → L2 冒烟 → L3 真实宿主	个可执行脚本 生成 / 校验 / 打包 / 漂移检测 / 装后自检	份参考资料 manifest 字典、权限全集、桥接协议、故障归因

1.1 它替你挡掉的坑

你本来会踩的坑	Skill 里的对应机制
manifest 少写 `main`，插件装不上	L0 静态检查直接报 `MANIFEST_MAIN_MISSING`
注册了 handler 却没注册 Action，点了没反应	L0「三件套配对」双向比对，缺一个就红
申请了权限但管理员没勾选，激活直接 500	L1 用 Proxy 复现隔离模式能力门禁，提前红
ZIP 里漏了 `storage/`，页面 404 但不报错	打包脚本强制校验 + 装后自检脚本逐项核对
用错命令命名空间，报 `No handler registered`	命名空间规则表 + 模板里前后端使用同一常量
中文正文 12px，投影上根本看不清	L0 `FONT_TOO_SMALL` 拦截（阈值 13px）

1.2 技能目录长什么样

```
~/.pi/agent/skills/openlearn-next-plugin-dev/
├── SKILL.md                    ← 主流程：六步法 + 红线
├── assets/                     ← 五套模板 + 测试护栏 + 缺陷报告模板
│   ├── server-only/  full-stack/  frontend-only/
│   ├── sandbox-view/           ← 第三方插件拿自定义界面的推荐模板
│   ├── inline-react/           ← 唯一可写 React 的模板（需宿主白名单）
│   └── test-harness/           ← L0/L1/L2 测试护栏源码
├── references/                 ← manifest 字典、权限全集、桥接协议、归因表
└── scripts/                    ← 可直接执行：生成 / 校验 / 打包 / 自检
    ├── new-plugin.sh               ← 一键生成项目骨架
    ├── facts-check.sh              ← 核对 skill 事实与平台源码是否漂移
    ├── platform-drift-check.sh     ← 平台事实漂移检测
    ├── post-install-check.sh       ← 装完之后的 10 项自检
    └── verify-templates-build.sh   ← 模板真实构建验证
```

一句话记住它的边界

Skill 负责「第三方 ZIP 插件」的全流程；**不负责**平台本体开发（packages/core）、 宿主内置插件、插件市场运营与发布流程，也不覆盖插件前端的深度定制（第三方没有自定义 React 组件通道，只有沙箱 `view`）。

SECTION 02

二、怎么让 AI 助手用上这个 Skill

2.1 三种触发说法

Skill 是靠语义触发的，不需要记命令。下面任一句话都能让助手加载它（`description` 里列出的关键词：开发 OpenLearn 插件、课堂工具、白屏、装不上…）。

```
# 说法一：直接开发
"帮我用 OpenLearn 插件开发技能做一个课堂投票插件"

# 说法二：从零开始
"我要开发一个第三方插件装到 OpenLearn 平台，请使用 openlearn-next-plugin-dev 技能"

# 说法三：排障（技能会自动切到归因模式）
"我上传的插件装上了但面板空白，日志里报 No handler registered for command type ..."
```

2.2 助手的标准工作流（Skill 的六步法）

STEP 1
探活 + 比对版本
确认平台版本、SDK 版本、端口、数据库、插件目录；跑 facts-check 看文档是否漂移。

STEP 2
需求 + P0 门禁
判插件类型、谁会看、UI 通道、执行模式、权限、落成 plugin.contract.json。

STEP 3
生成代码
选模板生成骨架，写 manifest 与业务逻辑，遵守 13 条编码护栏。

STEP 4
构建 + 测试护栏
构建产物预检清单；铺 L0–L2 护栏，L0 必须 0 error。

STEP 5
安装 + 验证
后台上传、勾选授权、激活、跑 post-install-check、L3 课堂冒烟。

STEP 6
交付或归因
按交付格式汇报；报错先分「插件侧 / 系统侧」再决定是否提 Issue。

2.3 你要提供什么，助手才能一次做对

你要说清	示例	助手据此产出
给谁用	「教师在讲台上投，学生在座位上看」	决定槽位：`teacher.tab` + `student.view`
要什么界面	「一个能点选项、能看到结果的页面」	通道判定：沙箱 `view`（HTML+桥接），而不是 React
要不要存数据	「投票结果要留下来，下次上课还能看」	插件类型：full-stack，建表 + 迁移
要不要 AI 参与	「老师说一句就能投」	注册 Action，让 AI Agent 可自然语言触发
目标宿主版本	「要能装到 0.7.10 的平台」	`engines.openlearn` 取实际用到的 API 最低版本

不要跳过的一句话

直接对助手说「**先别写代码，先帮我做 P0 门禁判定**」。 UI 通道一旦判错（最常见：以为要写 React 就去要白名单），后面所有工作都是白做。 判据见第 4 页。

SECTION 03

三、开工前的三件事

3.1 第一件：探活（30 秒，别猜）

端口、数据库路径、插件目录这三样缺一样，后面所有排查都会变成猜。 下面这段命令就是 Skill 的标准探活脚本，直接粘给助手或自己跑：

```
PLATFORM="${PLATFORM_REPO:-$HOME/Develop/openlearnv2}"
node -p "require('$PLATFORM/package.json').version"          # 平台版本
npm view @openlearn/plugin-sdk version                       # SDK latest（生成代码以它为准）
ss -ltnp 2>/dev/null | grep -E ':(9000|3000|5173)\b'         # 真实端口
ls -1 "$PLATFORM"/packages/core/db/*.db 2>/dev/null          # 数据库
ls -d "$PLATFORM"/plugins/*/ 2>/dev/null | head              # 已安装插件目录
curl -s localhost:9000/health                                # 服务是否就绪
```

```
$ node -p "require('$PLATFORM/package.json').version"
0.7.10
$ npm view @openlearn/plugin-sdk version
3.8.0
$ curl -s localhost:9000/health
{"status":"ok","uptime":36323,"version":"0.7.10"}
```

> **实测提醒：Skill 里的版本基线已经滞后**
>
> 本机实测平台是 **v0.7.10**、npm 上 SDK 是 **3.8.0**， 而 Skill 正文里记录的基线仍写着「平台 0.7.7 / SDK 3.7.5」。 这正是 Skill 把「每次开工先探活」列为第一步的原因： **版本号不能用来判断行为变化**，只能看源码与契约文件。 拿不准时以 `packages/core/` 、 `packages/plugin-sdk/` 的源码为准。

3.2 第二件：判断这件事是「写代码」还是「修 bug」

你说的话	助手进入的模式
「帮我做一个 XX 插件」	模式 A：开发（第一 ~ 第五步）
「插件装了但面板空白 / 点了没反应 / 报这个错」	模式 A'：先归因（插件侧 / 系统侧 / 配置 / 产品边界），再决定动作
「帮我处理 GitHub issue #123」	模式 B：仓库协作（占位 → worktree → PR）

3.3 第三件：确认目标宿主

插件里 `engines.openlearn` 写的是**宿主版本**（不是 SDK 版本）， 而且要按「实际用到的 API 的最低版本」填，不要一律抄 `>=0.2.5` ：

```
// 速查（来自 references/manifest-0.7.md §11）
只用 ctx.log / ctx.config            → engines.openlearn: '>=0.2.3'
用 ctx.http（api.routes）            → engines.openlearn: '>=0.3.11'
用声明式 contributes / 沙箱 view     → engines.openlearn: '>=0.7.0'
```

SECTION 04

四、第 1 步：一键生成项目骨架

不要手敲目录结构，用 Skill 自带的生成脚本。它比官方 CLI 多做两件事： 把 `{{commandPrefix}}` 这类占位符也一并替换掉（官方 CLI 只替换一部分）， 并且拒绝覆盖非空目录。

```
bash ~/.pi/agent/skills/openlearn-next-plugin-dev/scripts/new-plugin.sh \
  sandbox-view \
  ~/Develop/openlearn-plugins/classroom-wall \
  --id "@demo/classroom-wall" \
  --name "课堂便签墙" \
  --prefix "wall" \
  --description "教师在课堂中发布便签，学生端实时查看的课堂互动插件" \
  --author "OpenLearn 社区"
```

```
✓ 已生成 /home/wuxf/Develop/openlearn-plugins/classroom-wall（模板 sandbox-view）
  pluginId      @demo/classroom-wall
  commandPrefix wall
  view 文件     storage/*.html（安装器只解压 storage/**）

下一步：
  cd /home/wuxf/Develop/openlearn-plugins/classroom-wall
  cp -r .../assets/test-harness ./test-harness && npm i
  npm run pack                              # build + 补 storage/**
  node test-harness/static-check.mjs .      # L0: 先让它 0 error
```

4.1 五套模板怎么选

模板	适用	关键特征
`server-only`	只要 AI 工具 / 后端逻辑，不要界面	最小骨架，manifest + activate
`full-stack`	要后端命令 + 数据库（多数课堂插件）	多一个 `frontend.tsx` 桥接入口与 contracts
`frontend-only`	只要轻量前端能力（存储 / 提示）	无后端命令，注意能力声明为空时的门禁
`sandbox-view`	**第三方插件要自定义界面（推荐）**	HTML 页面 + 宿主注入的桥接脚本，**process 模式即可，不需要白名单**
`inline-react`	确实要写 React 组件（如白板画布卡片）	必须进宿主白名单，React 只能来自 `globalThis.HostSharedDeps`

> **选模板的第一判据是执行模式，不是功能多少**
>
> 第三方 ZIP 默认跑在 `process` 沙箱：没有 DOM、没有 React、没有网络。 只要不写 React， `sandbox-view` 就够了；只有真要自定义 React 组件才需要 `inline-react` ，并接受「必须进宿主白名单」这个前提。

4.2 生成后的目录

```
classroom-wall/
├── package.json          # 依赖 @openlearn/plugin-sdk
├── tsconfig.json         # 刻意不配 jsx（沙箱里没有 React）
├── plugin.contract.json  # P0 门禁结论落盘，L0/L2 会校验
├── scripts/pack.mjs      # 把 storage/** 补进官方 CLI 产出的 ZIP
├── src/
│   ├── index.ts          # manifest + activate: 后端逻辑
│   └── frontend.tsx      # 空桥接入口: 让 CLI 产出 frontend.js
└── storage/{teacher,student}.html    # 教师端 / 学生端界面
```

SECTION 05

五、第 2 步：P0 门禁，判 UI 通道

写代码前必须回答五个问题。答错一个，后面全是返工。判据写在 `plugin.contract.json` 里，L0/L2 会强制校验， **不要只在对话里口头约定。**

问题	怎么判	判错的后果
谁看？	教师 / 学生 / 两者。学生可见时必须有 `student.*` 槽位或白板挂件	学生端永远空白
要什么 UI？	宿主可渲染的声明式组件 / 自定义页面（`view`）/ 自定义 React 组件	白屏，且没有任何报错
执行模式？	第三方 ZIP 默认 `process`；自定义 React 组件必须 `inline`	process 里写 React → 激活即炸
凭什么 inline？	只有「文件位于 `packages/plugins`」或「id 在 `OPENLEARN_TRUSTED_INLINE_PLUGINS`」	以为 id 前缀带 `@openlearn/` 就能信任 → 被降级 process
数据怎么到学生端？	白板挂件必须派发 `whiteboard.draw(type:'plugin')`	只 `return { opened: true }`，学生端什么都没有

5.1 通道选择表

需要什么	用什么通道	执行模式
教师按钮 / Tab / 命令面板条目	`manifest.classroomTools` / `contributes`	process 即可
前端读后端数据、弹提示	桥接 `invokeCommand` / `api.*` + 轮询	process 即可
自定义界面（页面级）	`contributes.<slot>.view = 'storage/x.html'`（沙箱 iframe + 桥接）	**process 即可，无需白名单**
白板卡片（画布内 `type:'plugin'`）	配 `view: 'storage/xxx.html'`（推荐）；或 `registerExtensionPoint({component})`（仅 inline）	view 走 process / React 仅 inline
学生端浮层页面	优先 `view`，调 `bridge.ui.resize()` 自适应	process / inline

5.2 本次实例的判定结果

```
{
  "executionMode": "process",
  "uiChannel": "sandbox-view",
  "studentVisible": true,
  "whiteboardWidget": false,
  "slots": ["teacher.tab", "student.view"],
  "viewFiles": ["storage/teacher.html", "storage/student.html"]
}
```

浮层槽位的额外铁律

`student.classroom.overlay` 上的 `view` 必须固定定位（`fixed right-4 bottom-4 z-50`），**严禁** `w-full` / `h-full` / `flex-1`：它作为 Flex Item 会把白板挤成 0 宽， 现象是「插件面板正常、白板整块消失」。

SECTION 06

六、第 3 步：写后端（manifest 与「三件套」）

6.1 manifest：声明身份、版本、权限与入口

```
manifest: {
  id: '@demo/classroom-wall',          // 全局唯一，作者的身份
  name: '课堂便签墙',
  version: '0.1.0',
  main: 'index.js',                    // 必填！缺了装不上
  engines: { openlearn: '>=0.7.0' },  // 宿主版本，不是 SDK 版本
  requires: ['@openlearn/core:ICommandBusService@^1.0.0', '@openlearn/core:IActionRegistryService@^1.0.0',
             '@openlearn/core:IEventBusService@^1.0.0', '@openlearn/core:IDatabase@^1.0.0'],  // 按需声明
  capabilitiesProposed: ['lesson:read', 'lesson:write'],
  contributes: {
    'teacher.tab': [{ id: 'wall-teacher', label: '课堂便签墙',
                      view: 'storage/teacher.html' }],
    'student.view': [{ id: 'wall-student', label: '课堂便签墙',
                       view: 'storage/student.html' }],
  },
}
```

> **capabilitiesProposed 是「申请」，不是「授予」**
>
> 第三方插件声明的能力只是申请清单。管理员在安装向导里**勾选授予**后， 平台才会把能力发给你；不勾 = 授予集合为空 = 激活期任何非 get 调用都会被拒（ `Capability denied … only 'get' methods allowed` ）。

6.2 三件套：Action 描述符 + Handler + manifest 入口

内核对命令派发实行 **default-deny**：只有注册了 ActionDescriptor 的命令才会被派发。 每个 commandType 必须同时满足两件事，缺一即失败：

```
// ① ActionDescriptor：告诉内核「这个命令是干什么的、需要什么权限」
await actionRegistry.register({
  id: 'wall-add-note',
  commandType: 'wall.add_note',
  description: '在便签墙新增一条便签', capabilityRequired: 'lesson:write',
  inputSchema: {
    type: 'OBJECT',
    properties: {
      lessonId: { type: 'STRING', description: '课节 ID' },
      text:     { type: 'STRING', description: '便签内容' } },
    required: ['lessonId', 'text'] },
});

// ② Handler：必须是 { execute } 对象，不是裸函数
await commandBus.registerHandler('wall.add_note', {
  async execute(command) {
    // process 沙箱里 prepare()/run()/all() 返回 Promise，inline 下是同步值 —— 一律 await
    const rawDb = await (ctx.resolve as any)('@openlearn/core:IDatabase');
    const notesTable = ctx.db.table('notes');   // 自动带表名前缀
    await rawDb
      .prepare(`INSERT INTO ${notesTable} (id, lesson_id, text, author, created_at) VALUES (?, ?, ?, ?, ?)`)
      .run(row.id, row.lesson_id, row.text, row.author, row.created_at);
    return { id: row.id };
  },
});

// ③ manifest 入口：contributes / classroomTools 里引用的 commandType 必须与上面同名
```

SECTION 07

七、第 4 步：写可见界面（沙箱 view）

第三方插件的自定义界面 = 一个 HTML 文件 + 宿主注入的桥接脚本。 宿主用不透明源 iframe 加载 `/plugins/<id>/storage/xxx.html`， 并注入 `/plugin-sandbox/view-bridge.js`，页面通过 `window.OpenLearnPluginBridge` 调命令、接口、存储与提示。

```
<!-- storage/teacher.html -->
<script src="/plugin-sandbox/view-bridge.js"></script>   <!-- 必须引，否则没有桥接 -->
<script>
const bridge = window.OpenLearnPluginBridge;
let lessonId = null;

// ① 取上下文：宿主动态下发 lessonId / lang 等
bridge.onInit((ctx) => {
  lessonId = ctx.lessonId || null;
  refresh();
  setInterval(refresh, 3000);   // 沙箱内无推送，只能轮询
});

// ② 调后端命令：命令名必须与 registerHandler 的字符串完全一致
async function refresh() {
  const res = await bridge.invokeCommand('wall.list_notes', { lessonId });
  notesEl.innerHTML = (res?.notes ?? []).map(n => `<li>${esc(n.text)}</li>`).join('');
}
async function send() {
  await bridge.invokeCommand('wall.add_note', { lessonId, text: inputEl.value.trim() });
  await refresh();
}

// ③ 高度自适应：宿主按这个值撑开容器
new ResizeObserver(() => bridge.ui.resize(document.body.scrollHeight)).observe(document.body);
</script>
```

7.1 命令命名空间：最容易翻车的一处

平台的统一规则是：**凡是插件自己发出的命令，宿主都会用 `manifest.id` 加前缀。** 所以「注册端」和「调用端」必须写**同一个字符串：**

```
// 注册端
commandBus.registerHandler('wall.list_notes', { ... })
//   → 宿主存成 @demo/classroom-wall.wall.list_notes

// 调用端（沙箱页面）
bridge.invokeCommand('wall.list_notes', { ... })
//   → 解析成 @demo/classroom-wall.wall.list_notes  ✅ 对上了

// 如果调用端写成 'list_notes'
bridge.invokeCommand('list_notes', { ... })
//   → 解析成 @demo/classroom-wall.list_notes ✗ 404: No handler registered
```

本次实测踩到的坑

Skill 自带的 `sandbox-view` 模板里，注册端写的是 `{{commandPrefix}}.list_notes`，而页面里却写成了 `'list_notes'` —— 两者不一致，直接导致 404。已修复模板：页面统一改成 `{{commandPrefix}}.list_notes`。 **你自研插件时，请把命令名定义成单一常量，前后端共用。**

SECTION 07 · CONT.

七（续）：沙箱里的硬约束

不能做	原因与替代
`import React` / `document.` / `window.`（在 frontend.tsx 里）	process 沙箱跑在隐藏 iframe，没有 DOM / React；要界面就写 `view` 页面
`fetch()` / WebSocket	沙箱 CSP 是 `connect-src 'none'`；要走网络用后端 `ctx.http`
期望宿主推送事件	宿主不往 iframe 推事件；实时同步靠轮询 `invokeCommand`
`alert()`	用 `bridge.ui.showToast(msg,'error')`
裸模块说明符（`from 'x'`）	SDK CLI 3.7.2+ 构建期直接失败；依赖必须打进 bundle
自己 bundle 一份 React 渲染宿主组件	会触发 `Invalid hook call`；React 只能来自 `globalThis.HostSharedDeps`

身份敏感请求不要走 invokeCommand

沙箱命令通道强制使用「插件自己的 actor」，拿不到 `userId` / `role`。要认人，走 `bridge.api.*` → `ctx.http`（服务端从 `req.actor` 取身份）。

SECTION 08

八、第 5 步：铺测试护栏（L0–L3）

平台的 default-deny、能力校验、沙箱边界只在运行期生效，「能 build、能装上」≠「能用」。Skill 把这些坑做成了四层护栏，**L0 不过就不要进下一步。**

层	手段	耗时	抓什么
L0	`node test-harness/static-check.mjs .`	<1s	manifest 必填项 / 权限串真伪 / 隔离模式空能力门禁 / 三件套配对 / UI 通道契约 / 裸导入 / 中文 13px
L1	vitest `contract.test.ts` + `view-smoke.test.ts`	~0.3s	声明的入口与实际注册是否一致 / handler 形状 / activate 期间是否误调非 get RPC
L2	vitest `smoke.test.ts`	~0.3s	重复激活状态泄漏 / 命令执行期 TypeError / 事件 payload 形状 / 停用未退订
L3	真实宿主机：上传 → 激活 → 点入口 → 停用	3–10min	Mock 与生产宿主的差异（能力审批 / process 沙箱 / 熔断 / 全限定命令名）

8.1 铺设命令

```
cd ~/Develop/openlearn-plugins/classroom-wall
cp -r ~/.pi/agent/skills/openlearn-next-plugin-dev/assets/test-harness ./test-harness
npm i -D vitest @openlearn/plugin-test-kit better-sqlite3 uuid zod jsdom
npm pkg set scripts.test:contract="node test-harness/static-check.mjs ." \
            scripts.test="vitest run --config test-harness/vitest.config.ts"
```

两个必装依赖与一个易错点

`uuid` 与 `zod` 不是可选项：SDK 3.7.4 的 `dist/index.js` 引用了未声明的 `uuid`（`zod` 是 peerDep），不装会在 `import('@openlearn/plugin-sdk')` 时直接报 `Cannot find package 'uuid'`。

另外 `cp -r` 到**已存在**的目录会套娃成 `test-harness/test-harness/`，测试会以「找不到 `../src/index.js`」失败。复制前先 `rm -rf ./test-harness`。

8.2 实测输出

```
$ npm run test:contract

【L0 契约静态检查】.
  ⚠ [MANIFEST_SOURCE_ONLY] 未找到 manifest.json：按 src 源码推导 manifest（构建后再跑一次可获产物级校验）
  ✓ [CONTRACT_FOUND] 设计契约：plugin.contract.json

结果：0 error, 1 warn
```

```
$ npm test

 Test Files  3 passed (3)
      Tests  21 passed | 4 skipped (25)
   Duration  901ms
```

反向验证护栏本身

护栏没红过，就等于没有护栏。养成习惯：故意删掉一个 ActionDescriptor，确认 L0/L1 真的变红，再改回来。

SECTION 09

九、第 6 步：打包成可安装的 ZIP

官方 CLI 只打三个文件（`manifest.json` / `index.js` / `frontend.js`），`storage/**` 不会进包。而安装器**只解压 `storage/` 前缀的条目**，于是 view 页面在安装后 404 —— 但构建期零报错，运行期也只有「面板空白」。所以模板带了 `scripts/pack.mjs` 补这一步。

```
$ npm run pack

ℹ Building openlearn-plugin-classroom-wall-view...
✓ Bundled server + frontend
✓ Packaged openlearn-plugin-classroom-wall-view.zip (21.2 KB)
✓ 已重打包 openlearn-plugin-classroom-wall-view.zip（5 个文件，其中 2 个在 storage/ 下）
```

```
$ unzip -l openlearn-plugin-classroom-wall-view.zip
  Length      Date    Time    Name
 17665  2026-11-03 00:14   index.js
   944  2026-11-03 00:14   manifest.json
  2794  2026-11-03 00:14   frontend.js
     0  2026-11-03 00:14   storage/
  3401  2026-11-03 00:14   storage/student.html
  3869  2026-11-03 00:14   storage/teacher.html
```

9.1 ZIP 结构速查

条目	必需	说明
`manifest.json`	必需	必须在 ZIP 根目录；套一层目录会安装失败
`index.js`	必需	后端入口，名字要与 `manifest.main` 一致
`frontend.js`	建议	纯声明式 / 纯 view 插件可缺；但缺了会少一条注册路径，保守起见都带上
`storage/**`	用到 view 时必需	**必须保留 `storage/` 前缀**，否则安装器不认
`migrations/`	不需要	官方 CLI 不打包该目录；用 `ctx.db.migrate()` 做版本迁移

本次实测踩到的坑（已修模板）

Skill 自带 `pack.mjs` 原来把 `storage/` 下的文件按 **根级路径**写进 ZIP（`teacher.html` 而不是 `storage/teacher.html`），安装器因此完全不解压，页面直接 404（`ENOENT: no such file or directory ... /storage/teacher.html`）。模板已改为保留 `storage/` 前缀，并在打完包后校验「ZIP 里确有 storage/ 条目」，没有就直接退出非零。

安装前的最后一道人工预检

`manifest.main` 存在 · 每个 commandType 都有同名 ActionDescriptor · 前端产物无裸说明符 · `view` 以 `storage/` 开头且文件真在包里 · 中文正文 ≥ 13px。

SECTION 10

十、第 7 步：后台上传与授权

登录管理端 → 左侧「插件中心」→ 右上角「从 ZIP 安装」→ 选包 → 走完 5 步向导。 下面两张图是本次实拍的向导界面。

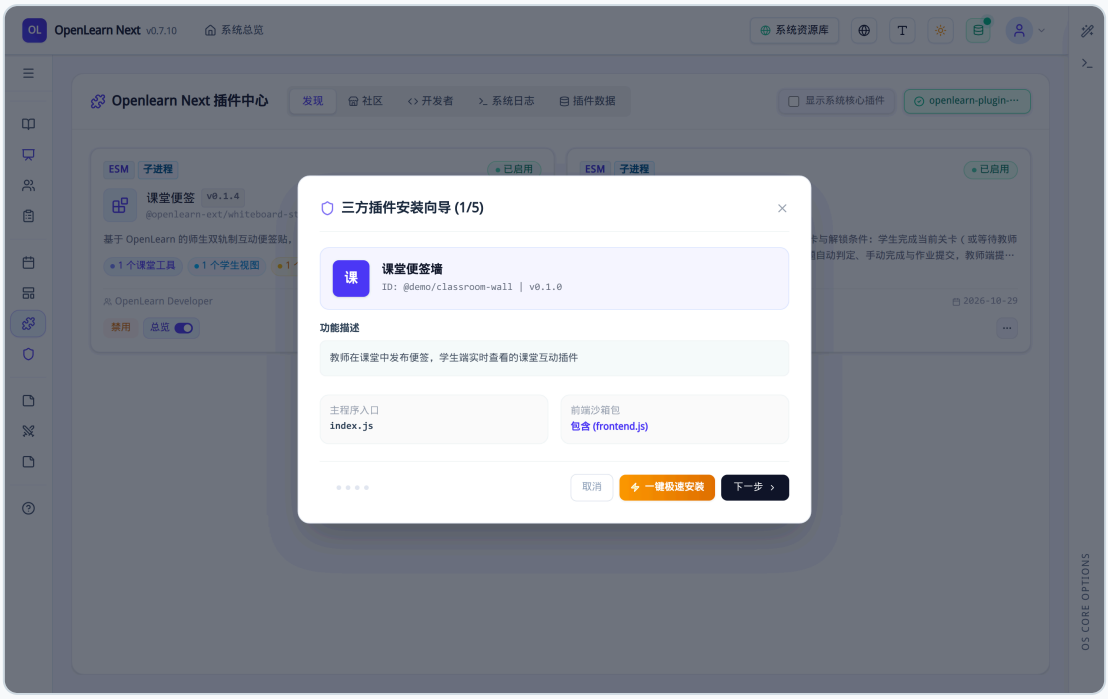


图 1 · 安装向导第 1 步。向导会先解析 manifest 并回显插件身份、描述、主程序入口与前端沙箱包； 若检测到同 ID 插件已安装，会转为「更新向导」并提示热更新可能导致课堂 UI 短暂异常。



图 2 · 第 2 步：权限申请与授权。插件声明的每一条能力都会列出风险等级。 **不勾选「我已了解并信任…」就没有任何授权**， 激活时任何非 get 调用都会被拒（`Capability denied … only 'get' methods allowed`）。

> **第三方插件必被强制为 process 模式**
>
> 无论 manifest 怎么写，第三方 ZIP 在平台上一律以 `process` 执行。 日志里看到 `[Plugin:<id>]` 前缀的输出是正常的； 如果激活失败，第一件事是看服务端日志，而不是反复改前端。

SECTION 11

十一、第 8 步：课堂验收与交付

11.1 装完立刻自检（别等页面报错）

安装成功后马上跑 Skill 的装后自检脚本。它一次核对十项：安装行状态、DB 里的 manifest 是否保住了每个 `view`、`view` 对应文件是否真落盘、ZIP 与 DB 是否一致。 这些失败在运行期**全部静默**（无异常日志，只有入口不出现）。

```
$ bash ~/.pi/agent/skills/openlearn-next-plugin-dev/scripts/post-install-check.sh \
    "@demo/classroom-wall" --zip ./openlearn-plugin-classroom-wall-view.zip

✓ 找到安装行 01a19ebe-6ed5-75f9-94b0-9cc7382452fb
  @demo/classroom-wall v0.1.0 status=active mode=process
✓ 状态 active
✓ DB 中的 manifest 声明了 2 个 view
  teacher.tab[0], student.view[0]
✓ teacher.tab[0].view 已落盘
  storage/teacher.html (3879 B)
✓ student.view[0].view 已落盘
  storage/student.html (3411 B)
✓ frontend.js 存在
  2794 B
✓ ZIP 共 6 个条目
✓ ZIP 内含 frontend.js
✓ ZIP 声明的 view 全部落进了 DB
✓ ZIP 内 view 文件齐全
▲ /api/plugins 返回 401
  需要登录态，跳过接口核对；静态资源仍可单独验

── 10 通过 / 1 提醒 / 0 失败
结论：结构完整。剩余 ▲ 多为环境提示，按 detail 处理。
```

11.2 入口出现在哪：扩展应用分组

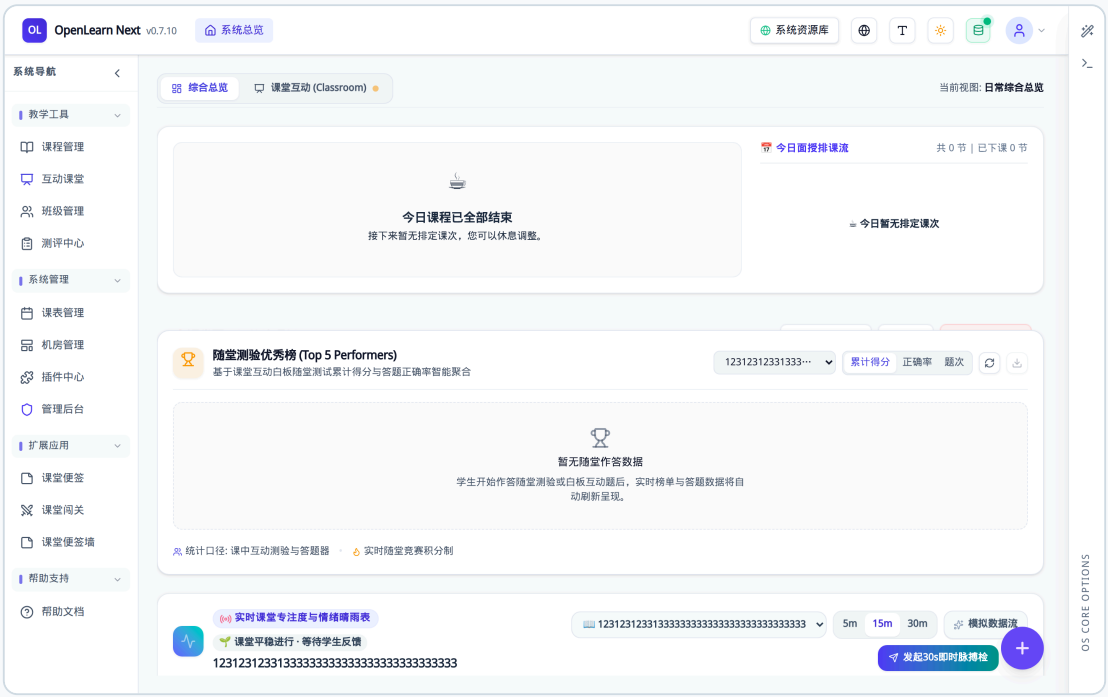


图 3 · 教师工作台左侧导航。插件声明的 `teacher.tab` 会自动出现在「扩展应用」分组里（图中「课堂便签」「课堂闯关」「课堂便签墙」三行均为插件贡献）， 不需要改宿主任何代码。

十一（续）：安装结果与插件实际运行效果

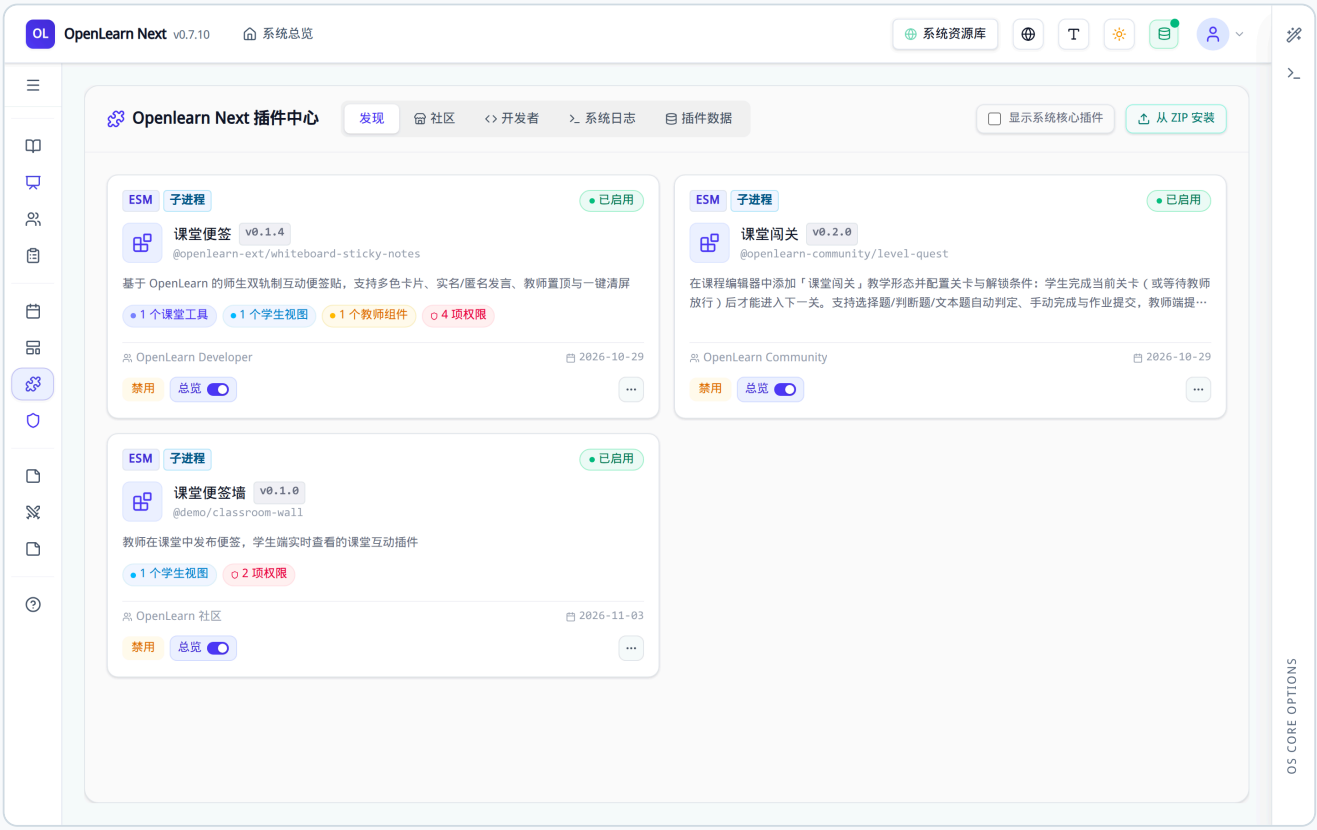


图 4 · 插件中心「发现」页。第三方插件以卡片形式出现，标注执行模式（ESM / 子进程）、 权限数量与贡献点数量；「禁用 / 总览」用于停用与查看详情。

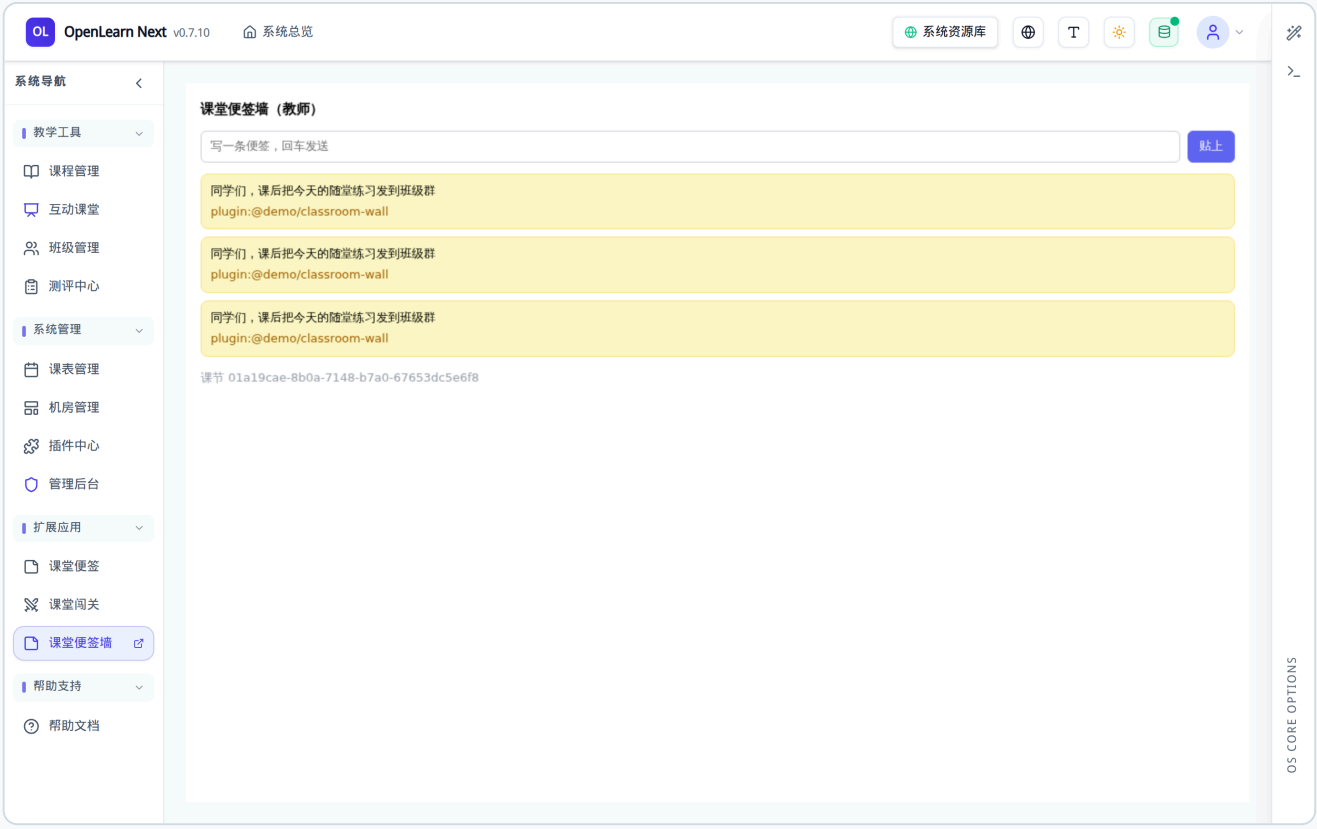


图 5 · 插件在教师端的真实运行界面。这是 `storage/teacher.html` 被宿主沙箱 iframe 加载后的样子：输入框、便签列表、当前课节 ID 全部来自桥接。 页面写的便签落进了插件自建表，刷新后仍在——说明 `invokeCommand` → handler → 数据库 整条链路是通的。

SECTION 12

十二、插件跑起来之后

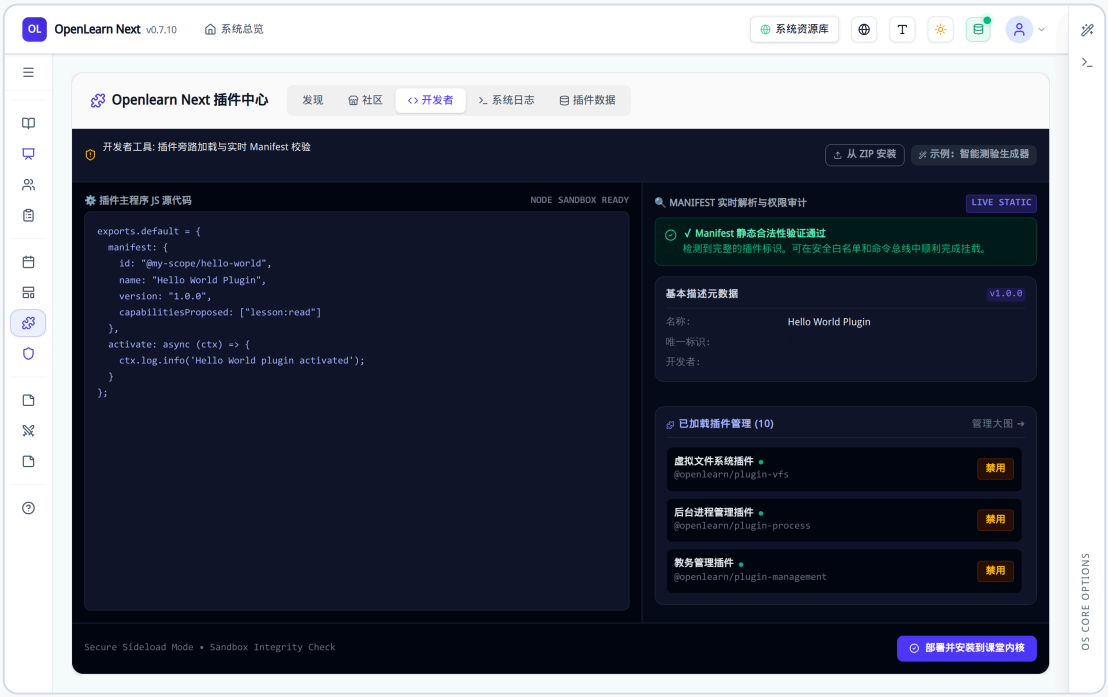


图 6 · 插件中心「开发者」页。内置的开发者工具可以旁路加载插件 JS 源码、 实时解析 manifest 并做权限合法性校验，适合快速验证 manifest 写法，不必每次都打包上传。

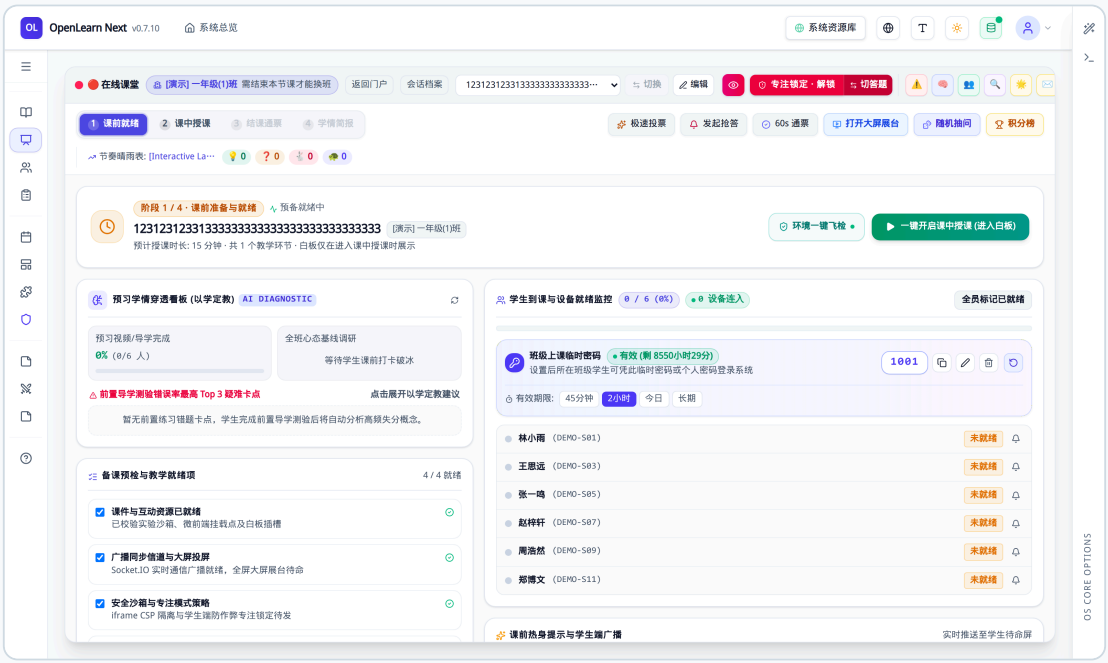


图 7 · 课堂现场。顶部工具条上的「极速投票 / 发起抢答 / 60s 通票 / 打开大屏展台 / 随机抽问 / 积分榜」等按钮，全部来自插件的 `classroomTools` 声明式贡献—— 插件不需要碰宿主代码，也不需要自己画按钮。

L3 真实宿主冒烟的 5 个必测点

① 课堂工具按钮点击有反应 · ② Ctrl+K 命令面板能派发（全限定命令名）· ③ AI 自然语言能触发 Action · ④ 停用再激活无重复订阅 · ⑤ `view` 页面 200、桥接脚本 200、 `bridge.invokeCommand` 通、 `ui.resize` 生效。

SECTION 13

十三、踩坑清单：本次实测翻车的四个点

这四条不是理论推演，是用 Skill 的模板从零做出「课堂便签墙」时真实踩到的， 而且其中两条就藏在 Skill 自带模板里。

症状	真正的原因	修复
面板显示 `ENOENT … /storage/teacher.html`，白屏无日志	ZIP 里 `storage/` 前缀丢了，文件被写成根级条目，安装器不解压	打包时保留 `storage/` 前缀；装后跑自检脚本核对
`No handler registered for command type "@demo/classroom-wall.list_notes"`	调用端与注册端命令名不一致（注册 `wall.list_notes`，调用 `list_notes`）	命令名定义成单一常量，前后端共用；或调用时写全 `wall.list_notes`
`#<Promise> could not be cloned`	handler 返回了 Promise：process 沙箱里 `prepare().all()` 是异步的，模板漏了 `await`	所有 DB 调用一律 `await`（inline 同步、process 异步都兼容）
vitest 报「找不到 `../src/index.js`」	`cp -r` 套娃成 `test-harness/test-harness/`	复制前先 `rm -rf ./test-harness`

13.1 高频报错速查（先归因，再动手）

报错 / 现象	归属	机制层缘由
`× frontend 产物里有裸模块说明符`（构建失败）	插件侧	沙箱 iframe 用 Blob + 动态 import 加载，无 importmap，裸说明符无从解析
`COMMAND_NOT_REGISTERED` / 403	插件侧	内核 default-deny：该 commandType 没有 ActionDescriptor
`Capability denied … only 'get' methods allowed`	配置	0.7.2 起按**已授予**集合判定：声明了但管理员没勾
`Import of "x" is not allowed`	插件侧	token-enforcer 拦截裸导入（`crypto` / `fs` / 第三方裸包）
`Cannot read properties of undefined (reading 'React')`	插件侧	远程前端用了 React，沙箱内没有 `HostSharedDeps`
`DataCloneError`（传 `component`）	产品边界	postMessage 无法克隆函数；远程插件没有自定义 React 组件通道
插件被自动停用	产品边界	熔断器：5 分钟滑窗内 3 次异常即停用，属设计行为
装上了但入口不出现	多因	manifest 字段被 zod strip / view 未落盘 / 权限没授予 / frontend.js 404

> **判断「系统侧还是插件侧」的四个必要条件**
>
> ① 官方脚手架原始产物或 ≤50 行最小插件能稳定复现 · ② 版本号、复现步骤、完整堆栈齐全 · ③ 报错点位于宿主 / 内核 / SDK 范围内 · ④ 已排除版本不匹配、产物没重建、浏览器缓存、 权限未审批、被强制 process 等因素。缺任何一条，只能判「待定」，不要去提 Issue。

SECTION 14

十四、交付清单与更新 / 卸载

14.1 Skill 定义的「完成」标准

	验收项
□	构建成功，产出 ZIP；`manifest.json` 通过平台 schema 校验
□	每个 commandType 都有同名 ActionDescriptor
□	`capabilitiesProposed` 覆盖 activate 期所有非 get RPC，且安装时已勾选授予
□	`post-install-check.sh` 0 失败
□	可见入口按槽位类型分别验证（页面级 / 浮层 / 白板卡片 / 行内按钮）；中文正文 ≥ 13px
□	`engines.openlearn` 与实际用到的 API 匹配
□	L0 静态契约检查 0 error；L1/L2 全绿；L3 真实宿主冒烟通过

14.2 助手的交付汇报格式

```
【插件】@demo/classroom-wall · sandbox-view · engines >=0.7.0
文件：manifest.json / src/index.ts / src/frontend.tsx / storage/{teacher,student}.html
构建：npm install && npm run pack  →  上传 openlearn-plugin-classroom-wall-view.zip
已实现：教师端贴便签、学生端查看、按课节持久化、事件广播
测试：L0 0 error · L1/L2 21 passed · L3 已跑（教师端面板 + 写便签 + 回读）
已核对：main / ActionDescriptor 配对 / 权限串 / 可见入口 / 中文字号 ≥ 13px
自检：post-install-check 10 通过 / 0 失败
待办：管理员在安装向导勾选能力授予（lesson:read / lesson:write）
```

14.3 更新与卸载

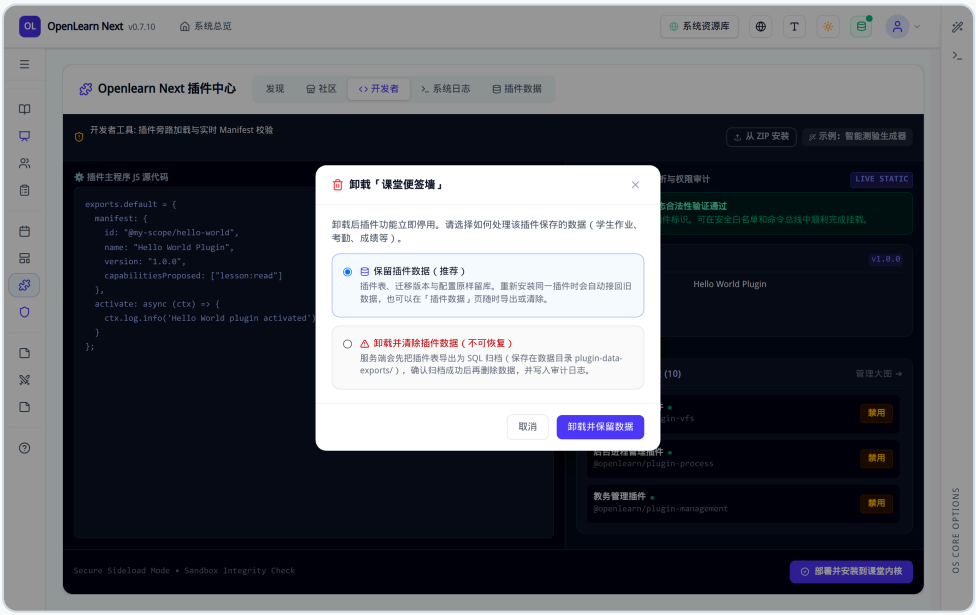


图 8 · 卸载时的数据处置。「保留插件数据」会把插件表与迁移版本导出到 `plugin-data-exports/`，重新安装同一插件时自动接回旧数据；「卸载并清除」不可恢复，只写审计日志。开发迭代时选前者。

> **迭代时的推荐节奏**
>
> 改代码 → `test:contract` + `npm test` → `npm run pack` → 后台「从 ZIP 安装」（同 ID 走更新向导）→ 勾选授权 → `post-install-check.sh` → 课堂点一遍。 热更新时正在上课的 UI 可能短暂异常，选课间时段更新最稳。

APPENDIX

附录：资料索引与术语

A.1 该去哪查什么

我想知道…	去哪里
manifest 每个字段怎么写	Skill `references/manifest-0.7.md`；平台 `docs/plugin/plugin-manifest-spec.md`
平台到底有哪些能力字符串	Skill `references/capability-matrix-0.7.md`；平台 `docs/agent/contracts/`
沙箱桥接能调什么、不能调什么	Skill `references/frontend-bridge-0.7.md`；平台 `docs/plugin/frontend-sandbox-capability-boundary.md`
四类槽位分别怎么渲染	平台 `docs/reference/plugin-ui-extension-slots.md` §1.1.1
报错该怎么归因	Skill `references/troubleshooting.md`；平台 `docs/beta/known-issues.md`
文档与源码冲突时听谁的	源码 > `contracts/*.json` / facts-ledger > 平台 docs > Skill 快照

A.2 本手册涉及的核心术语

术语	含义
manifest	插件的身份说明书：id、版本、入口、权限申请、贡献点
capabilitiesProposed	插件申请的系统能力清单（申请 ≠ 授予）
contributes	声明式贡献：往宿主槽位里放页面、按钮、视图
view	沙箱 HTML 页面（`storage/xxx.html`），第三方拿自定义界面的推荐方式
三件套	manifest 入口 + ActionDescriptor + registerHandler，三者命令名必须一致
process / inline	两种执行模式：process 是子进程沙箱（第三方默认），inline 是宿主内联（需白名单）
L0–L3	四层测试护栏：静态契约 / 契约与沙箱冒烟 / 冒烟 / 真实宿主

下一步建议

先用 `server-only` 模板做一只 30 行的「Hello World」插件跑通全链路（探活 → 生成 → 打包 → 上传 → 激活 → 卸载），再回到 `sandbox-view` 做界面。 分两步走，两次踩坑的定位难度完全不同。